

第2章

直接刚度法

2.4 平衡方程组的求解

Solution of Equilibrium Equations



2.4 平衡方程组的求解

□ 直接解法 $\mathbf{K}\mathbf{a} = \mathbf{R}$

- 高斯消去法
- LDL^T 分解 ($\mathbf{K} = \mathbf{L}\mathbf{D}\mathbf{L}^T$)
- Cholesky 分解 ($\mathbf{K} = \mathbf{L}\mathbf{L}^T$)
- 波前法

可事先准确地估算出求解方程组所需要的算术运算次数

□ 迭代解法

- Jacobi迭代法
- Gauss-Seidel 迭代法
- 预处理共轭梯度法

收敛所需的运算次数取决于矩阵 $\mathbf{K}$ 的条件数和加速方案，无法事先准确估算

□ 计算量：CPU时间 = Cn^α

- n : 自由度总数
- C 和 α : 取决于所使用的求解方法、矩阵的稀疏性和条件数

✓ 直接解法:

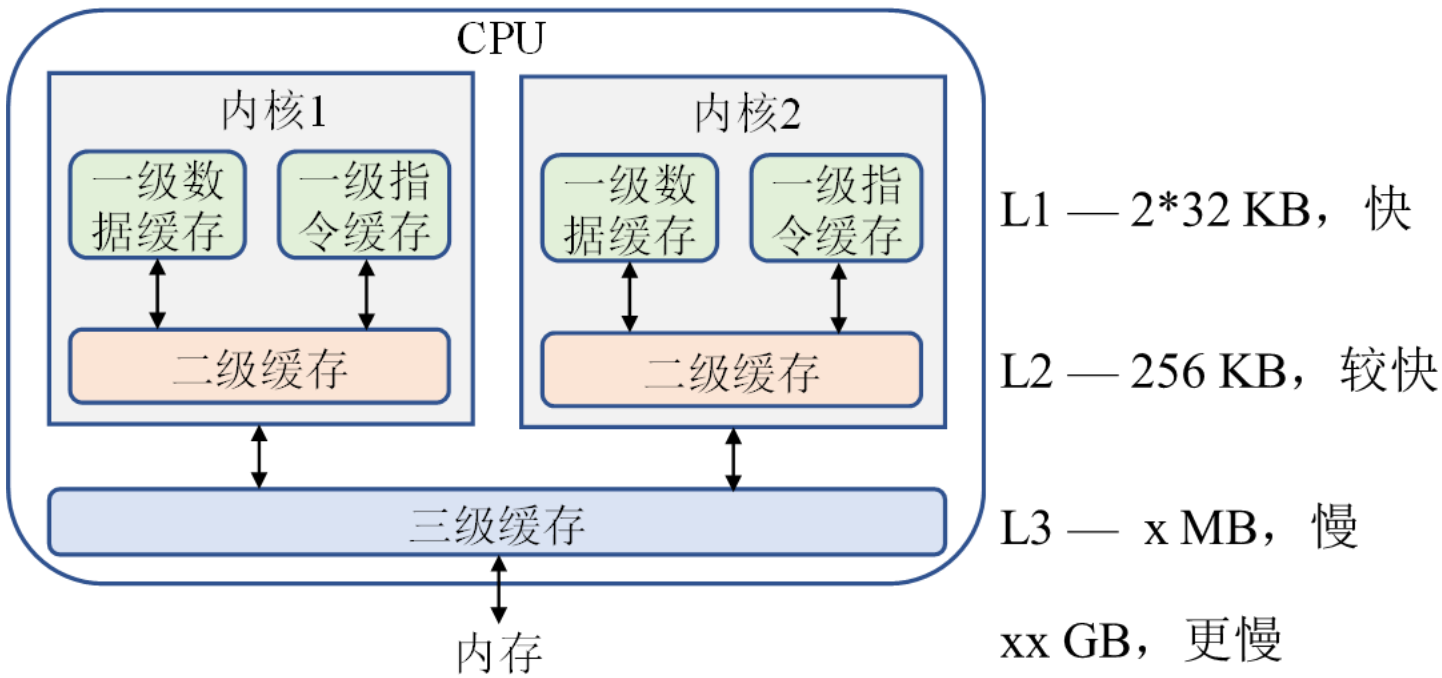
- 稠密矩阵 $\alpha = 3$; 若求解 10^3 阶方程组需要1s的话, 求解 10^6 阶方程组则需要 10^9 s (~30年)!
- 稀疏矩阵 $\alpha = 1 \sim 2$ ($\beta^2 n$)

✓ 迭代解法: $C_{\text{ite}} \gg C_{\text{dir}}, \alpha_{\text{ite}} < \alpha_{\text{dir}}$

- 对于大型问题, 迭代法的效率可能要比直接法高得多

2.4 平衡方程组的求解

CPU缓存体系



- 缓存命中(cache hit): CPU所要操作的数据已经在缓存中, 可直接读取, 否则(cache miss)需先将其从内存载入到缓存
- 缓存命中会带来很大的性能提升
- 对 LDLT 方法进行程序实现时, 应优化代码以尽可能提升 CPU 缓存的命中率

CPU三级缓存

计算机的内存是一维的, 多维数组被转换为一维存储:

- 按行: C/C++和python语言
1, 4, 5, 0, -14, -14, 0, 0, -8
- 按列: FORTRAN和matlab语言
1, 0, 0, 4, -14, 0, 5, -14, -8

$$\begin{bmatrix} 1 & 4 & 5 \\ 0 & -14 & -14 \\ 0 & 0 & -8 \end{bmatrix}$$

在循环体中应尽量按顺序连续地操作同一块内存上的数据, 以提升CPU缓存命中率

```
for (i=0; i<N; i+=1)
    for (j=0; j<N; j+=1)
        a[i][j]=0;
```

2.4 平衡方程组的求解

LDL^T 求解

高斯消元法 (Gaussian Elimination)

$$\begin{bmatrix} 1 & 4 & 5 \\ 4 & 2 & 6 \\ 5 & 6 & 3 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

- 将第1行乘以-4和-5分别加到第2行和第3行

$$\begin{bmatrix} 1 & 4 & 5 \\ 0 & -14 & -14 \\ 0 & -14 & -22 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

- 将第2行乘以-1加到第3行

$$\begin{bmatrix} 1 & 4 & 5 \\ 0 & -14 & -14 \\ 0 & 0 & -8 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

高斯消元等价于对矩阵进行初等变换
第三类初等行变换：将矩阵的第*i*行乘以常数*c*加到第*j*行上所对应的初等矩阵记为 $T_{ij}(c)$

初等行变换等价于用初等矩阵左乘矩阵

$$\mathbf{K} = \begin{bmatrix} 1 & 4 & 5 \\ 4 & 2 & 6 \\ 5 & 6 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 5 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 4 & 5 \\ 0 & -14 & -14 \\ 0 & 0 & -8 \end{bmatrix}$$

消去第1列：

$$\mathbf{K}_1 = \mathbf{T}_{13}(-5)\mathbf{T}_{12}(-4)\mathbf{K}$$

消去第2列：

$$\begin{aligned} \mathbf{K}_2 &= \mathbf{T}_{23}(-1)\mathbf{K}_1 \\ &= \mathbf{T}_{23}(-1)\mathbf{T}_{13}(-5)\mathbf{T}_{12}(-4)\mathbf{K} \end{aligned}$$

矩阵LU分解：

$$\begin{aligned} \mathbf{K} &= \mathbf{T}_{12}(4)\mathbf{T}_{13}(5)\mathbf{T}_{23}(1)\mathbf{K}_2 \\ &= \mathbf{L}\mathbf{U} \quad \mathbf{U} = \mathbf{K}_2 \text{ — 上三角阵} \\ \mathbf{L} &= \mathbf{T}_{12}(4)\mathbf{T}_{13}(5)\mathbf{T}_{23}(1) \text{ — 下三角阵} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 5 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \end{aligned}$$

l_{ji} ：在消去第*i*列非对角元素时从第*j*行中减去第*i*行所乘的系数

$$\mathbf{T}_{ij}(c) = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & \cdots & 0 \\ & & \vdots & \ddots & \vdots \\ & & \textcircled{c} & \cdots & 1 \\ & & & \ddots & \\ & & & & 1 \end{bmatrix}$$

(j, i)

$$\mathbf{T}_{ij}^{-1}(c) = \mathbf{T}_{ij}(-c)$$

$$\mathbf{T}_{12}(-4) = \begin{bmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad l_{21} = 4$$

$$\mathbf{T}_{13}(-5) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -5 & 0 & 1 \end{bmatrix} \quad l_{31} = 5$$

$$\mathbf{T}_{23}(-1) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix} \quad l_{32} = 1$$

2.4 平衡方程组的求解

$$K = LU = LD\tilde{U} \quad \tilde{U} = D^{-1}U$$

矩阵 L 的元素 l_{ji} 等于在消去矩阵 K 第 i 列非对角元素时从第 j 行中减去第 i 行所乘的系数

$$K^T = \tilde{U}^T D L^T \quad \text{若 } K \text{ 对称: } \tilde{U} = L^T \quad K = \overset{??}{L} D L^T$$

$$K = LU \quad k_{ij} = \sum_{r=1}^i l_{ir} u_{rj} = \sum_{r=1}^{i-1} l_{ir} u_{rj} + u_{ij} \quad (i=1:j)$$

$$U = D L^T \quad u_{ij} = d_{ii} l_{ji}$$

$$l_{ji} = u_{ij} / d_{ii} \quad u_{ij} = k_{ij} - \sum_{r=1}^{i-1} l_{ir} u_{rj} \quad (i=1:j-1)$$

$$d_{jj} = u_{jj} = k_{jj} - \sum_{r=1}^{j-1} l_{jr} u_{rj}$$



练习：根据递推公式计算 l 、 u 、 d

$$\begin{bmatrix} 1 & 4 & 5 \\ 4 & 2 & 6 \\ 5 & 6 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 5 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 4 & 5 \\ 0 & -14 & -14 \\ 0 & 0 & -8 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 5 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -14 & 0 \\ 0 & 0 & -8 \end{bmatrix} \begin{bmatrix} 1 & 4 & 5 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

LDL^T 求解

$$L = \begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & \ddots & & \\ \vdots & \vdots & \vdots & 1 & \\ l_{n1} & l_{n2} & \cdots & l_{n,n-1} & 1 \end{bmatrix}$$
$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & \cdots & u_{1n} \\ & u_{22} & u_{23} & \cdots & u_{2n} \\ & & \ddots & \cdots & \vdots \\ & & & u_{n-1,n-1} & u_{n-1,n} \\ & & & & u_{nn} \end{bmatrix}$$
$$D = \begin{bmatrix} u_{11} & & & & \\ & u_{22} & & & \\ & & \ddots & & \\ & & & & u_{nn} \end{bmatrix}$$
$$L^T = \begin{bmatrix} 1 & l_{21} & l_{31} & \cdots & l_{n1} \\ & 1 & l_{32} & \cdots & l_{n2} \\ & & \ddots & \cdots & \vdots \\ & & & 1 & l_{n,n-1} \\ & & & & 1 \end{bmatrix}$$

2.4 平衡方程组的求解

$$Ka = R \xrightarrow{\text{分解}} K = LDL^T$$

$$LDL^T a = R$$

$$LV = R \xrightarrow{\text{消去}} \sum_{r=1}^{i-1} l_{ir} v_r + v_i = r_i$$

$$v_i = r_i - \sum_{r=1}^{i-1} l_{ir} v_r \quad (i = 1:n)$$

消去

$$DL^T a = V \xrightarrow{\text{回代}} L^T a = D^{-1} V = \bar{V}$$

$$a_i + \sum_{r=i+1}^n l_{ri} a_r = \bar{v}_i \quad \bar{v}_i = \frac{v_i}{d_{ii}}$$

$$a_i = \bar{v}_i - \sum_{r=i+1}^n l_{ri} a_r \quad (i = n:1)$$

回代

- 对右端项 R 的消去过程既可以在对矩阵 K 进行分解的同时进行，也可以在矩阵 K 完成分解后进行
- 为高效求解多载荷工况问题，有限元程序一般都是先对矩阵 K 进行分解，然后再对多个不同的右端项进行消去与回代求解

$$L = \begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & \ddots & & \\ \vdots & \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \cdots & l_{n,n-1} & 1 \end{bmatrix}$$

$$L^T = \begin{bmatrix} 1 & l_{21} & l_{31} & \cdots & l_{n1} \\ & 1 & l_{32} & \cdots & l_{n2} \\ & & \ddots & \ddots & \vdots \\ & & & 1 & l_{n,n-1} \\ & & & & 1 \end{bmatrix}$$

LDL^T 求解

$$\begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 5 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -14 & 0 \\ 0 & 0 & -8 \end{bmatrix} \begin{bmatrix} 1 & 4 & 5 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 5 & 1 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \xrightarrow{\text{消去}} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -14 & 0 \\ 0 & 0 & -8 \end{bmatrix} \begin{bmatrix} 1 & 4 & 5 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 4 & 5 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -1/8 \end{bmatrix} \quad L^T a = D^{-1} V$$

$$\begin{cases} a_3 = \bar{v}_3 = -\frac{1}{8} \\ a_2 = \bar{v}_2 - l_{32} a_3 = \frac{1}{8} \\ a_1 = \bar{v}_1 - l_{21} a_2 - l_{31} a_3 = \frac{1}{8} \end{cases}$$

2.4 平衡方程组的求解

活动列解法

- ❖ 程序实现
 - 尽可能提高方程组求解器的效率
 - 尽可能减少其内存需求量以提高可求解问题的规模
 - 具有外存求解(out-of-core solution)能力

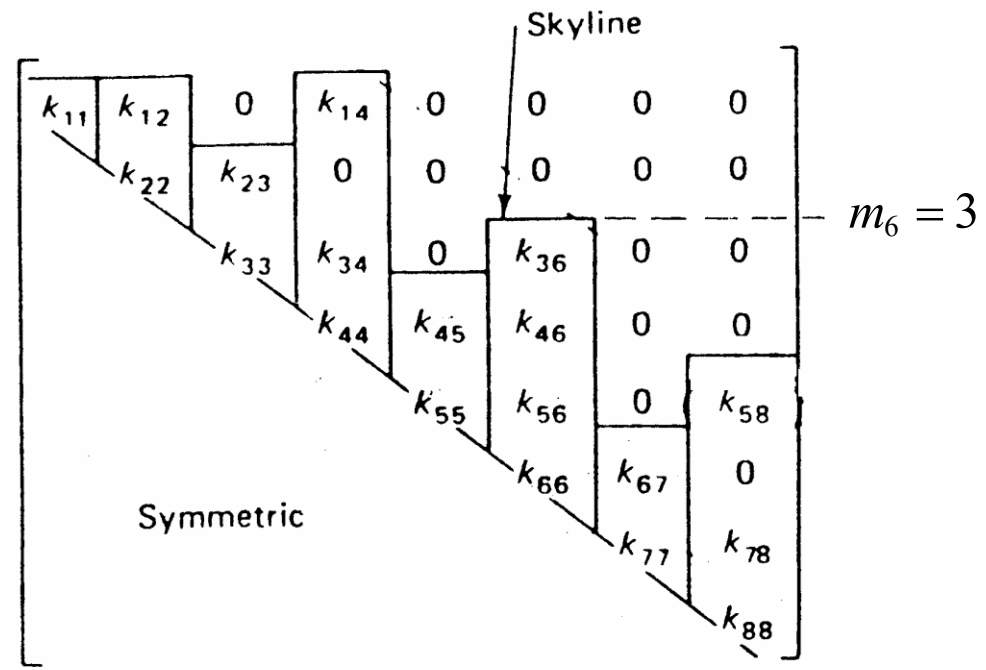
- ❖ 活动列解法 (轮廓/列消去法)
 - LDL^T 分解可以按列依此进行
 - 第*j*列 (*j* = 2:*n*) 元素 *u_{ij}* 和 *d_{jj}* 的计算

$$d_{11} = u_{11} = k_{11}$$
$$u_{ij} = k_{ij} - \sum_{r=1}^{i-1} l_{ir} u_{rj} \quad i = 2:j-1$$
$$= k_{ij} - \sum_{r=1}^{i-1} L_{ri} u_{rj}$$
$$l_{ji} = u_{ij} / d_{ii} \quad L_{ij} = u_{ij} / d_{ii}$$
$$d_{jj} = u_{jj} = k_{jj} - \sum_{r=1}^{j-1} l_{jr} u_{rj} = k_{jj} - \sum_{r=1}^{j-1} L_{rj} u_{rj}$$

$m_j + 1$

$\max(m_i, m_j)$

m_j



在对矩阵*K*的第*j*列进行分解时，需要用到矩阵*L*的第*i*行和矩阵*U*的第*j*列各元素



- ✓ 产生的后果? cache miss !
- ✓ 解决方案?

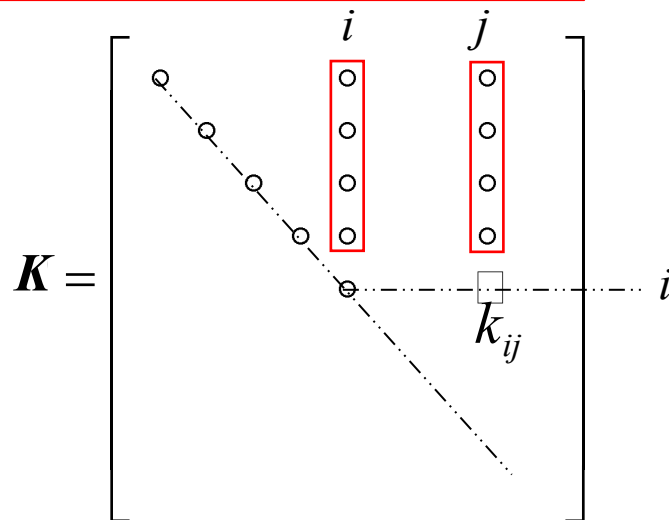
将对矩阵*L*的*i*行操作改为对矩阵 *L^T* 的第*i*列操作!

$l_{ir} \longrightarrow l_{ir} = (L^T)_{ri} = L_{ri}$

活动列解法 (轮廓/列消去法)

$$j = 2:n, \quad i = m_j + 1:j-1$$

$$\begin{aligned} d_{11} &= k_{11} \\ u_{ij} &= k_{ij} - \sum_{r=m}^{i-1} L_{ri} u_{rj} \\ m &= \max(m_i, m_j) \\ L_{ij} &= u_{ij} / d_{ii} \\ d_{jj} &= k_{jj} - \sum_{r=m_j}^{j-1} L_{rj} u_{rj} \end{aligned}$$



➤ 内存复用

$$\diamond k_{ij} \leftarrow L_{ij} \quad i = m_j + 1, \dots, j-1$$

$$\diamond k_{jj} = d_{jj}$$

半带宽优化 — 尽可能减小半带宽以减小计算量

➤ 所需浮点运算次数

$$\frac{1}{2} \sum_{j=1}^n (j - m_j)^2 \leq \frac{1}{2} n M_K^2 \quad M_K = \max_{1 \leq j \leq n} (j - m_j) \text{ — 最大半带宽}$$

➤ 外存求解？

➤ 分块求解 — 分块组装消去

➤ 波前法 (Bruce Irons, 1970) — 组装和消去交替进行；内存大于最大波前区就可求解；内外存交换频繁

➤ **MUMPS** (MUltifrontal MAssively Parallel sparse direct Solver)

<https://mumps-solver.org/>

➤ **PARDISO** (PARallel DIrect SOLver) ★

<http://www.pardiso-project.org>

Intel **MKL** (Intel Math Kernel Library)

https://scc.ustc.edu.cn/zlsc/user_doc/html/intel-mkl/intel-mkl.html

2.4 平衡方程组的求解

活动列解法

```
def colsol(n,m,K,R):  
    IERR = 0  
  
    # Perform L*D*L(T) factorization of stiffness matrix  
    for j in range(1, n):  
        for i in range(m[j]+1, j):  
            c = 0.0  
            for r in range(max(m[i],m[j]), i):  
                c += K[r,i]*K[r,j]  
  
            K[i,j] -= c  
  
        for r in range(m[j], j):  
            Lrj = K[r,j]/K[r,r]  
            K[j,j] = K[j,j] - Lrj*K[r,j]  
            K[r,j] = Lrj  
  
        if K[j,j] <= 0:  
            print('Error - stiffness matrix is not positive definite !')  
            print('      Nonpositive pivot for equation ', j)  
            print('      Pivot = ', K[j,j])  
  
        IERR = j  
        return IERR, K, R  
    ... ..
```

colsol.py

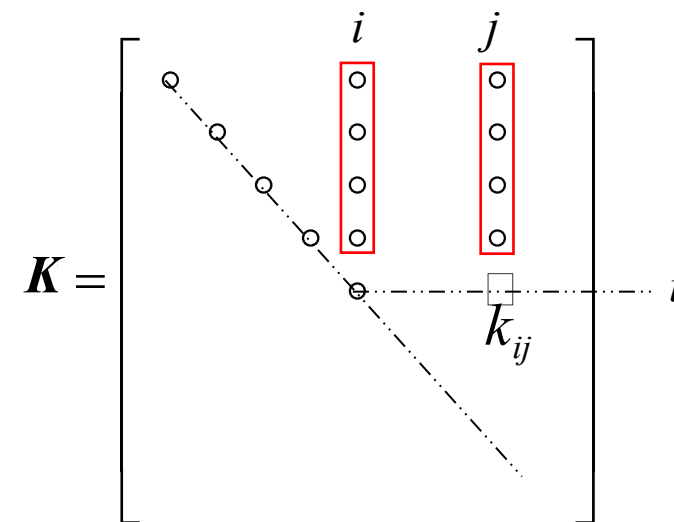
矩阵非正定（零主元）

colsol-python

<https://github.com/xzhang66/FEM-Book>
FEM-Python/colsol-python

$$j = 2:n, \quad i = m_j + 1:j-1$$

$$\begin{aligned} d_{11} &= k_{11} \\ u_{ij} &= k_{ij} - \sum_{r=m}^{i-1} L_{ri} u_{rj} \\ m &= \max(m_i, m_j) \\ d_{jj} &= k_{jj} - \sum_{r=m_j}^{j-1} L_{rj} u_{rj} \\ L_{rj} &= u_{rj} / d_{rr} \end{aligned}$$



2.4 平衡方程组的求解

$LV = R$

$v_1 = r_1$
 $v_i = r_i - \sum_{r=1}^{i-1} l_{ir} v_r \quad (i = 2:n)$
 $= r_i - \sum_{r=m_i}^{i-1} L_{ri} v_r$

```
# Reduce right-hand-side load vector
for i in range(1, n):
    for r in range(m[i], i):
        R[i] -= K[r,i] * R[r]

for i in range(0, n):
    R[i] /= K[i,i]

# Back-substitute
for i in range(n-1, 0, -1):
    for r in range(m[i], i):
        R[r] -= K[r,i]*R[i]

return IERR, K, R
```

colsol.py

$L^T a = \bar{V} \qquad \bar{V} = D^{-1}V$

$a_n = \bar{v}_n$
 $a_i = \bar{v}_i - \sum_{r=i+1}^n l_{ri} a_r \quad (i = n-1:1)$
 $= \bar{v}_i - \sum_{r=i+1}^n L_{ir} a_r$

- 计算 a_i 时需要用到 L^T 第 i 行从第 $i+1$ 列到第 n 列的元素 $\Rightarrow$ cache miss
- 为提高缓存命中率，对 L^T 的各列 ($i = n:2$) 循环，依次计算 L^T 的各列对 a_i 的贡献：

第 i 列贡献：
 $a_r^{(i-1)} = a_r^{(i)} - L_{ri} a_i^{(i)}, r = m_i : i-1$
最终解：
 $a_{i-1} = a_{i-1}^{(i-1)}$

活动列解法

所需浮点运算次数：

$2 \sum_{i=1}^n (i - m_i) \leq 2nM_K$

$\begin{bmatrix} 1 & 4 & 5 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -1/8 \end{bmatrix}$

$\begin{cases} a_3 = \bar{v}_3 = -\frac{1}{8} \\ a_2 = \bar{v}_2 - L_{23} a_3 = \frac{1}{8} \\ a_1 = \bar{v}_1 - L_{12} a_2 - L_{13} a_3 = \frac{1}{8} \end{cases}$

第3列： $a_1^{(2)} = a_1^{(3)} - L_{13} a_3^{(3)}$
 $a_2^{(2)} = a_2^{(3)} - L_{23} a_3^{(3)}$

第2列： $a_1^{(1)} = a_1^{(2)} - L_{12} a_2^{(2)}$

最终： $a_1 = a_1^{(1)}, a_2 = a_2^{(2)}, a_3 = a_3^{(3)}$

2.4 平衡方程组的求解

活动列解法

```
import json
import numpy as np
from colsol import colsol

# Read in equations from JSON file
with open('Example_n_5.json') as f_obj:
    Equations = json.load(f_obj)

n = Equations['n']
m = np.array(Equations['m'])
K = np.array(Equations['K'], dtype='f')
R = np.array(Equations['R'], dtype='f')

[IERR, K, R] = colsol(n, m, K, R)

if IERR==0:
    print("K = \n", K)
    print("R = \n", R)
```

$$\begin{bmatrix} 2 & -2 & 0 & 0 & -1 \\ -2 & 3 & -2 & 0 & 0 \\ 0 & -2 & 5 & -3 & 0 \\ 0 & 0 & -3 & 10 & 4 \\ -1 & 0 & 0 & 4 & 10 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$



Example_n_5.json

```
{
    "n": 5,
    "m": [0, 0, 1, 2, 0],
    "K": [[ 2, -2, 0, 0, -1],
          [-2, 3, -2, 0, 0],
          [ 0, -2, 5, -3, 0],
          [ 0, 0, -3, 10, 4],
          [-1, 0, 0, 4, 10]],
    "R": [ 0, 1, 0, 0, 0 ]
}
```

□ 浮点数是以二进制形式存储的

- 单精度(32位): 23位尾数位, 相邻浮点数间距为 $2^{-23} = 1.29 \times 10^{-7}$ (机器误差)
- 双精度(64位): 52位尾数位, 相邻浮点数间距为 $2^{-52} \approx 2.22 \times 10^{-16}$ (机器误差)

□ 初始截断误差(δK 和 δR) / 求解过程舍入误差

$$(K + \delta K)\bar{a} = (R + \delta R) \quad \bar{a} = a + \delta a$$

$$\delta a = (K + \delta K)^{-1}[R + \delta R - (K + \delta K)a]$$

$$= (I + K^{-1}\delta K)^{-1} K^{-1}[\delta R - \delta K a]$$

$$\| (I + K^{-1}\delta K)^{-1} \| \leq \frac{1}{1 - \| K^{-1} \| \cdot \| \delta K \|} \quad (\text{设 } \| K^{-1} \| \cdot \| \delta K \| < 1)$$

$$\| R \| = \| K a \| \leq \| K \| \cdot \| a \| \Rightarrow \frac{1}{\| a \|} \leq \frac{\| K \|}{\| R \|}$$

$$\| \delta a \| \leq \frac{\| K^{-1} \|}{1 - \| K^{-1} \| \cdot \| \delta K \|} \left(\frac{\| \delta K \|}{\| K \|} \| K \| \cdot \| a \| + \frac{\| \delta R \|}{\| R \|} \| K \| \cdot \| a \| \right)$$

$$\frac{\| \delta a \|}{\| a \|} \leq \frac{\text{cond}(K)}{1 - \| K^{-1} \| \cdot \| \delta K \|} \left(\frac{\| \delta K \|}{\| K \|} + \frac{\| \delta R \|}{\| R \|} \right)$$

$\text{cond}(K) = \| K \| \cdot \| K^{-1} \|$ — 条件数

$$\text{cond}(K)_2 = \lambda_n / \lambda_1$$

- 若 $\| \delta K \|^$ 和 $\| \delta R \|^$ 均为小量, $\| \delta a \|^$ 是否也是小量?
- 若 $\| K \bar{a} - R \|^$ 为小量时, $\| \delta a \|^$ 是否也是小量?

方程 $Ka = R$ 的残差向量为:

$$r = K\bar{a} - R = K\delta a$$

$$\| \delta a \| = \| K^{-1} r \| \leq \| K^{-1} \| \cdot \| r \|^$$

$$\frac{\| \delta a \|}{\| a \|} \leq \text{cond}(K) \frac{\| r \|}{\| R \|}$$

- 对于良态问题, 方程组残差向量范数 $\| r \|^$ 可以用来度量数值解的精度
- 对于病态问题, 即使方程组残差向量的范数 $\| r \|^$ 比较小, 数值解的相对误差仍然可能较大

- 条件数可以看成是相对误差的放大倍数
- 对于病态问题, 条件数 $\text{cond}(K)$ 很大, 很小的截断误差 δK 和 δR 都可能会使数值解产生较大的误差。

利用高斯消元法求解线性方程组

$$\begin{bmatrix} 0.913 & 0.659 \\ 0.457 & 0.330 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 0.254 \\ 0.127 \end{bmatrix}$$

此问题的精确解为: $[a_1 \ a_2] = [1 \ -1]^T$

系数矩阵的条件数为: 12485.0

利用高斯消元法进行消去运算, 在运算过程中仅保留小数点后4位, 得

$$\begin{bmatrix} 0.9130 & 0.6590 \\ 0 & 0.0002 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 0.2540 \\ 0.0001 \end{bmatrix}$$

回代可解得

$$[a_1 \ a_2] = [-0.0827 \ 0.5]^T \quad \text{方程残差为 } [r_1 \ r_2] = [-5.1 \times 10^{-6} \ 2.061 \times 10^{-4}]^T$$

如果采用python的float16 (半精度, 3~4位有效数字), 其解为

$$[a_1 \ a_2] = [0.639 \ -0.5]^T \quad \text{方程残差为 } [r_1 \ r_2] = [0.0 \ 1.221 \times 10^{-4}]^T$$

$$\frac{\|\delta \mathbf{a}\|}{\|\mathbf{a}\|} \leq \text{cond}(\mathbf{K}) \frac{\|\mathbf{r}\|}{\|\mathbf{R}\|} \quad \leftarrow \begin{bmatrix} a_1 & a_2 \end{bmatrix} = [-0.0827 \ 0.5]^T$$

(3.6503)

(9.0638)

$$\frac{\|\mathbf{r}\|}{\|\mathbf{R}\|} = 7.260 \times 10^{-4}$$

与舍入误差同量级! $\Rightarrow a_2$ 本质上是任意的!

残差小, 但解的精度低!